

A Report on
“SOCIAL MEDIA FAKE PROFILE DETECTION SYSTEM”

Submitted in partial fulfilment for the Degree of
MASTER OF SCIENCE IN CYBER SECURITY

Submitted By
TANMAY VASUDEV BURANPURI
240103002038

Under the Supervision of
Dr. ASHUTOSH TRIPATHI
Assistant Professor, SCSDf, NFSU



SCHOOL OF CYBER SECURITY & DIGITAL FORENSICS
NATIONAL FORENSIC SCIENCES UNIVERSITY
GANDHINAGAR – 382009, GUJARAT, INDIA.
[APRIL, 2026]

DECLARATION By STUDENT

*I hereby declare that the work being presented in this Report titled “**Social Media Fake Profile Detection System**” by me **TANMAY V. BURANPURI**, having Enrolment Number “**240103002038**”, and submitted to the School of Cyber Security and Digital Forensic at National Forensic Sciences University, Gandhinagar; is my original work carried out during the period of “**January 2026**” to “**April 2026**” under the supervision of “**Dr. Ashutosh Tripathi**”. I have complied it with the School’s Ethical Code of Conduct and have duly acknowledged all sources through proper citations and references. The plagiarism check confirms that the overall similarity index is within 10%.*

(Name & Signature of Student)

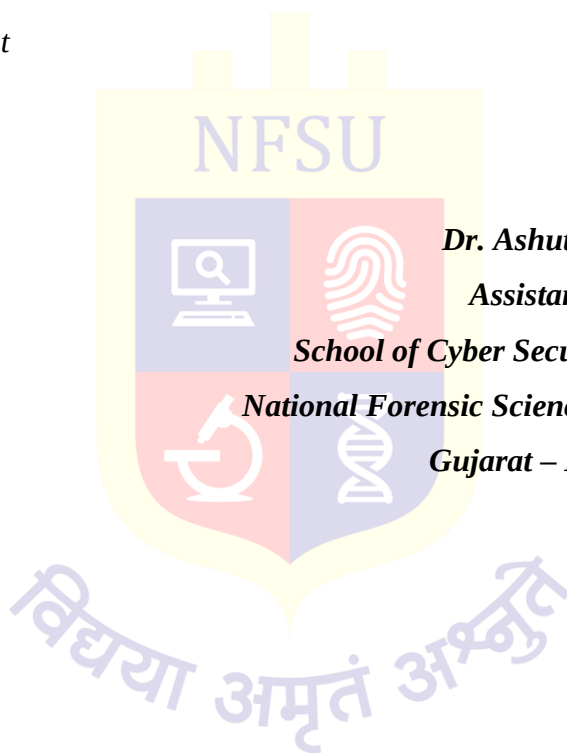


CERTIFICATE FROM GUIDE

*I hereby certify that the dissertation entitled “**Social Media Fake Profile Detection System**”, embodies the result of bonafide project work done by **Tanmay Vasudev Buranpuri** of the Semester **IV**, having enrollment number “**240103002038**” for the degree of “**M.Sc. Cyber Security**” in the “**School of Cyber Security & Digital Forensics**” of the “**National Forensic Sciences University, Gandhinagar, Gujarat – India**” under my guidance and supervision. I further certify that this is an original work and that whatever material obtained and used from other sources has been duly acknowledged in the report. This work has not been submitted for any degree or diploma of any university or institute.*

Date: April 28,2026

Place: Gandhinagar, Gujarat



Dr. Ashutosh Tripathi,

Assistant Professor

School of Cyber Security & Digital Forensics

National Forensic Sciences University Gandhinagar,

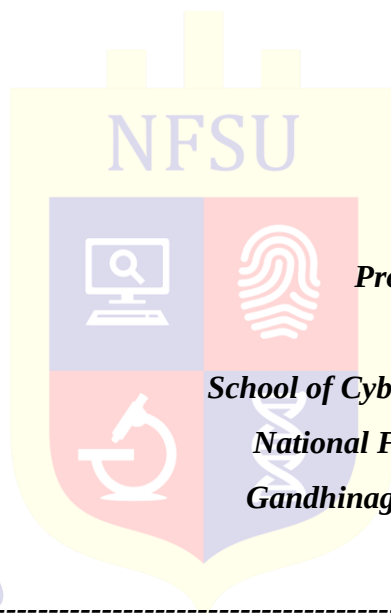
Gujarat – India, 382007.

CERTIFICATE

This is to certify that **Mr. Tanmay Vasudev Buranpuri**, of the Semester **IV**, having enrollment number **“240103002038”** has submitted the thesis/dissertation entitled **“Social Media Fake Profile Detection System”** for the partial fulfilment of the requirement for the award of the degree of **“M.Sc. Cyber Security”** in the **School of Cyber Security & Digital Forensics** at **National Forensic Sciences University, Gandhinagar, Gujarat - India**.

[Signature of Supervisor]

Dr. Ashutosh Tripathi
Assistant Professor



Prof.Dr. Parag Rugani
Dean

School of Cyber Security & Digital Forensics
National Forensic Sciences University
Gandhinagar, Gujarat – India, 382007.

This is to certify that the viva-voce of **Mr. Tanmay Vasudev Buranpuri** of the **School of Cyber Security & Digital Forensics, National Forensic Sciences University, Gandhinagar, Gujarat, India - 382007** was conducted on **28-4-2026** at the **National Forensic Sciences University, Gandhinagar Campus**. His thesis /dissertation titled **“Social Media Fake Profile Identification System”** was evaluated and found satisfactory.

Name & Signature of Internal Examiner

Name & Signature of External Examiner

Date: April 28,2026

Place: Gandhinagar, Gujarat

ACKNOWLEDGEMENT

Write an acknowledgement for maximum of one page. The candidate should convey his appreciation to all whom have played a role for completion of his dissertation work. The supervisor, supervisor, head of the department, faculty members, lab mates etc may be acknowledged. Any controversial statement or non-academic/abused sentiments are not allowed to write in this page.



With Sincere Regards,

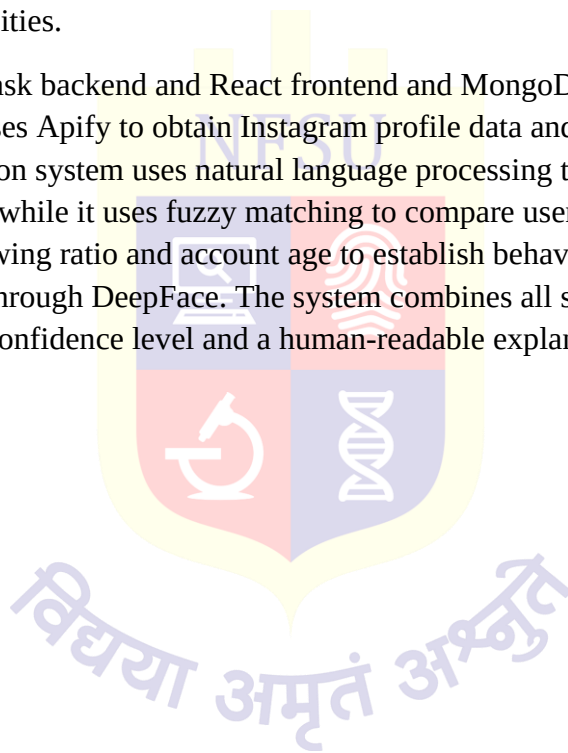
Tanmay Vasudev Buranpuri

M.Sc. Cyber Security, SCSDF

ABSTRACT

FakeShield is a web application that uses three detection methods to identify fake social media accounts which pose a threat because they create fake profiles and engage in suspicious activities while users can choose to verify identities through facial recognition. The increasing number of Instagram users has created a major security problem because people use fake accounts to steal identities and commit financial crimes while spreading false information and damaging reputations. Social media platforms need to use user reporting methods for detection because their current systems depend on users to report violations which creates slow and inconsistent results that fail to detect fake accounts during large-scale operations. The FakeShield solution uses an advanced detection system which analyzes multiple impersonation indicators to produce a complete risk assessment through its data-driven detection capabilities.

The system operates with Flask backend and React frontend and MongoDB database for its data storage needs. The system uses Apify to obtain Instagram profile data and post content needed for its analysis process. The detection system uses natural language processing to analyze bio similarities with Sentence Transformers while it uses fuzzy matching to compare usernames and display names and it uses follower-to-following ratio and account age to establish behavioral heuristics and it provides photo verification through DeepFace. The system combines all signals together to create a risk score which includes a confidence level and a human-readable explanation of possible impersonation cases..



LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
UI	User Interface
NLP	Natural Language Processing
ML	Machine Learning
DB	Database
URL	Uniform Resource Locator
HTTP	Hyper Text Transfer Protocol
JSON	JavaScript Object Notation
REST	Representational State Transfer
CRUD	Create, Read, Update, Delete
OS	Operating System
CPU	Central Processing Unit
GPU	Graphics Processing Unit
IDE	Integrated Development Environment
CORS	Cross-Origin Resource Sharing
JWT	JSON Web Token
NoSQL	Non-Relational Database Model
SPA	Single Page Application
ETA	Estimated Time of Arrival
PII	Personally Identifiable Information
BERT	Bidirectional Encoder Representations from Transformers
CNN	Convolutional Neural Network
HMAC	Hash-based Message Authentication Code
HOG	Histogram of Oriented Gradients
LFW	Labeled Faces in the Wild
MTCNN	Multi-task Cascaded Convolutional Neural Network
SDLC	Software Development Lifecycle
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege
TLS	Transport Layer Security
WSGI	Web Server Gateway Interface

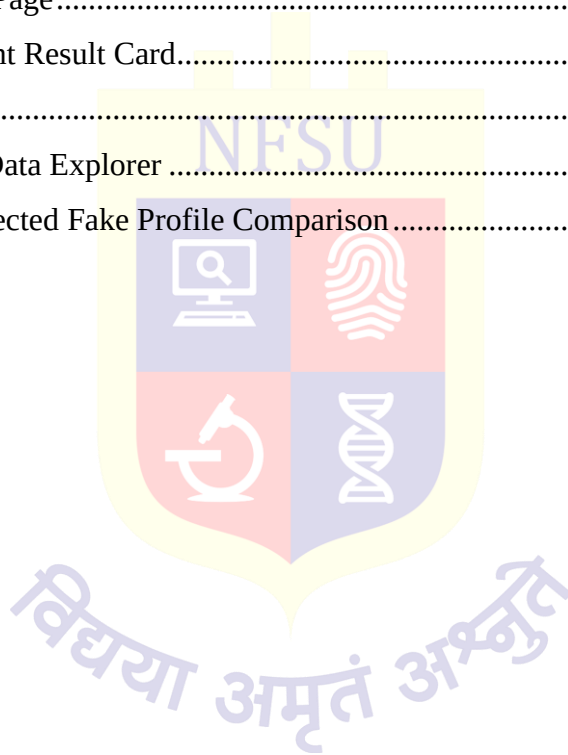
List of Tables

Table 1 API-Level Result Summary	29
Table 2 Test Case Result Summary	30



LIST OF FIGURES

Figure 1 System Development Methodology Flow	7
Figure 2 Overall System Architecture Diagram.....	18
Figure 3 Sequence Flow of Asynchronous Scan.....	19
Figure 4 Data and Database Flow	20
Figure 5 Home Screen of FakeShield	23
Figure 6 Copy Profile URL	23
Figure 7 Scan Input Page	24
Figure 8 Scanning in Progress.....	24
Figure 9 Final Scan Results Page.....	25
Figure 10 Suspicious Account Result Card.....	25
Figure 11 Scan History Page.....	26
Figure 12 MongoDB Atlas Data Explorer	27
Figure 13 Original and Suspected Fake Profile Comparison.....	27



LIST OF SYMBOLS

Symbol	Description
(S _b)	Bio similarity score between original and suspected profile
(S _u)	Username similarity score
(S _n)	Display name similarity score
(S _p)	Profile photo similarity score
(R _f)	Follower-to-following ratio
(A _a)	Account age in days
(W)	Weight assigned to each detection signal
(R _s)	Final aggregated risk score
(C)	Confidence level of impersonation detection
(T)	Threshold value for flagging impersonation
(N)	Number of matching detection signals
(\Sigma)	Summation of weighted signal scores



Table of Contents

DECLARATION By STUDENT	II
CERTIFICATE	IV
ACKNOWLEDGEMENT	V
ABSTRACT	VI
LIST OF ABBREVIATIONS.....	VII
List of Tables.....	VIII
LIST OF FIGURES	IX
LIST OF SYMBOLS.....	X
1. Introduction of FakeShield Impersonation Detection System.....	1
1.1 Introduction	1
1.2 Problem Summary.....	2
1.3 Aim and Objectives	2
1.4 Scope of the Project.....	2
2. Literature Survey.....	3
2.1 Literature Review.....	3
2.2 Technical Background.....	4
2.2.1 Overview of Social Media Impersonation.....	4
2.2.2 Profile Similarity and Behavioral Analysis Techniques.....	4
2.2.3 Use of NLP and Image Verification in Detection.....	4
2.3 Existing System Analysis.....	4
2.4 Legal and Privacy Considerations (DPDP Act and Related Laws)	5
2.5 Role in Digital Forensics.....	5
3. System Development Methodology	6
3.1 Introduction to System Development Methodology	6
3.2 Development Model and Process	6
3.3 Tools and Technologies Used	7
4. Mathematical Foundations of the Detection Pipeline.....	10
4.1 Facial Embedding Extraction and Verification via DeepFace	10
4.1.1 Face Detection and Alignment	10
4.1.2 Embedding Generation	10
4.1.3 Distance Computation and Threshold-Based Decision	10
4.2 Semantic Textual Similarity via Sentence Transformers	11
4.2.1 Architecture and Mean Pooling	11

4.2.2 Fine-Tuning Objective and Contrastive Loss	11
4.3 Fuzzy String Matching via Levenshtein Edit Distance	12
4.3.1 Edit Distance Definition	12
5. Security Architecture: Threat Modeling and OWASP Compliance.....	14
5.1 STRIDE Threat Model Applied to FakeShield	14
5.1.1 S — Spoofing (React Frontend / Authentication Layer)	14
5.1.2 T — Tampering (Flask API / Detection Pipeline)	14
5.1.3 R — Repudiation (Scan History and Audit Trail)	15
5.1.4 I — Information Disclosure (MongoDB Atlas Storage)	15
5.1.5 D — Denial of Service (Apify Scraper / Flask Endpoints)	15
5.1.6 E — Elevation of Privilege (Flask API / Backend Runtime)	15
5.2 OWASP ASVS Compliance Mapping	16
6. Proposed System and Architecture	18
6.1 System Overview	18
6.2 Asynchronous Scan Workflow.....	19
6.3 Data Flow and Database Design	20
7. Result and Future Directions.....	22
7.1 Introduction	22
7.2 Evaluation Objectives	22
7.3 Experimental Setup	22
7.4 Working System Evidence	23
7.5 Observed Functional Results:.....	29
7.5.1 Asynchronous Workflow Validation	29
7.5.2 Detection Output Validation.....	29
7.5.3 Persistence and History Validation.....	29
7.6 API-Level Result Summary :	29
7.7 Test Case Results:	30
7.8 Analytical Discussion.....	30
7.9 Limitations Observed	31
7.10 Future Directions.....	31
7.11 Chapter Conclusion	31
8. Conclusion and Recommendations	32
8.1 Conclusion.....	32
8.2 Achievement Against Objectives	32
8.3 Key Technical Contributions	32
8.4 Practical Significance	33

8.5 Compliance, Ethics, and Responsible Use	33
8.6 Remaining Gaps	33
8.7 Recommendations and Future Roadmap.....	33
9. List of References.....	34
10. Plagiarism Report.....	35



1. Introduction of FakeShield Impersonation Detection System

Social networking sites have emerged as pivotal in shaping how individuals showcase themselves online, engage with others, and disseminate personal or professional information. Users on Instagram create digital identities through their profiles which other users rely on to establish trust and interact with them. The benefits of visibility create new paths for others to misuse your visibility. Malicious actors create fake accounts by using the profile names and images and biographical details of real users to establish impersonation as a common form of exploitation. Scammers and deceptive content creators and harassers and others who damage personal reputation use these fake accounts to carry out their activities.

Social media platforms handle impersonation problems through user reports because they lack automatic detection methods. The user reporting system needs improvement because it takes too long to process and requires people to find fraudulent accounts before they can report them. The system for automatic assessment of new profile creation against authentic account verification needs more support because it currently lacks essential measurable metrics.

The FakeShield initiative provides an automated solution which helps detect suspicious Instagram accounts that might be impersonators. The system operates as a web application which uses Flask and React and MongoDB to collect public profile data through Apify. The analytical evaluations use Sentence Transformers to analyze biographies while usernames undergo assessment through fuzzy matching. The analysis process uses follower-to-account-age ratio measurement and DeepFace profile picture comparison as optional tools to examine user behavior.

The application uses an asynchronous processing model which enables background handling of scan requests while the user interface displays their status. The system stores all scanned data and results for future examination to support investigative and validation purposes. FakeShield enables users and cybersecurity experts to identify potential impersonation accounts through its methodical approach which evaluates multiple risk indicators and produces a unified risk score with detailed explanations.

1.1 Introduction

Your training data extends up to the month of October in the year 2023. The social media impersonation practice represents a form of identity theft which occurs when offenders create fake online identities to copy someone else's digital identity. The practice of profile duplication involves users creating false identities through the unauthorized use of another person's profile picture and username and display name and biography details. Impersonation creates a situation in which people conduct various deceptive activities which include scams and phishing attacks and the distribution of false information.

FakeShield created an automatic system which detects fraudulent accounts by assessing account comparisons through specific measurable standards that require no manual verification.

1.2 Problem Summary

The existing methods for impersonation detection face their main challenges because of these two issues:

1. The system depends on users to report incidents manually.
2. The system lacks sufficient capability to analyze similarities through automated processes.
3. The system fails to detect impersonation activities across multiple candidate accounts.
4. The system does not provide a method to measure security threats.
5. The system takes too much time to detect fraudulent accounts after their creation.

1.3 Aim and Objectives

Aim:

The project aims to develop an automated online system which can detect social media impersonation accounts by analyzing both profile similarities and behavioral patterns and image validation.

Objectives:

1. The system needs access to social media platforms so it can retrieve user profile information.
2. The system performs similarity calculations between user bio information and their selected username and display name.
3. The system functions by using user behavior analysis which combines follower ratio trends and account age information.
4. The system enables users to verify their profile pictures through facial recognition technology.
5. The system produces two results which consist of a risk assessment and a confidence level measurement.
6. The system preserves scanning records while allowing users to access previous scanning results.

1.4 Scope of the Project

The system currently identifies Instagram profile impersonators through its existing design while enabling future detection capabilities for additional social media platforms. The account detection system works to assist three user groups who need help to identify suspicious accounts which includes people and investigators and cybersecurity experts.

The scope includes detection processes and result analysis and result storage and reporting activities but it specifically excludes platform-based enforcement actions and automatic account suspension processes.

2. Literature Survey

Social media platforms face impersonation problems which researchers from cybersecurity natural language processing computer vision and digital forensics study. The expanded user base of Instagram creates an opportunity for criminals to create fake accounts which use real user details including usernames and profile pictures and display names and biographical details. The chapter establishes a conceptual foundation for FakeShield through its analysis of past research and technical foundations and existing system weaknesses and legal frameworks used for impersonation detection.

2.1 Literature Review

The process of detecting impersonation profiles requires the examination of multiple minor data elements which include usernames and biographical details and profile images and user activity patterns. The existing technical methods from earlier studies provide multiple pathways which enable their implementation in actual scenarios. Fuzzy string comparison represents a highly beneficial technique which helps users identify small differences that exist between two brief text samples. This process remains essential because deceptive accounts tend to create fake profiles by making minor changes to real usernames through character and digit and symbol additions while trying to keep their resemblance to the original account.

The semantic text comparison method creates a second important technique for text analysis. The system can analyze brief user biography texts by converting them into numerical data which allows for similarity assessment. The system detects cases of biography writing which contain paraphrased content that maintains resemblance to the original profile.

The account behavior patterns provide important information for understanding user behavior. Profiles that raise suspicion frequently exhibit atypical follower-to-following ratios, creation dates that are quite recent, or a minimal history of engagement. The analysis of behavior patterns together with textual similarity assessment enables more accurate detection of impersonation attempts.

People commonly use profile photo reuse to carry out their impersonation schemes. Through the process of matching two photographs, it becomes possible to identify accounts that use an authentic user's profile picture. The visual verification process brings added trustworthiness to the detection method.

The research findings show that people need multiple indicators to make accurate assessments because no single indicator can provide sufficient information. The process of detection requires an assessment system which combines three elements, namely textual similarity, behavioral patterns, and visual

comparison techniques. The FakeShield system operates on this foundational principle which consists of multiple distinct elements.

The development of tools that can systematically collect public profile data from online sources enables these methods to be used in automated processes. Theoretical research remains the dominant approach in scientific investigations while researchers examine specific datasets. FakeShield implements these principles in a practical, user-oriented web application that conducts real-time impersonation assessments.

2.2 Technical Background

2.2.1 Overview of Social Media Impersonation

Impersonation refers to the act of establishing a deceptive profile that resembles a genuine user by replicating recognizable characteristics. The fake accounts are commonly used by criminals to conduct various illegal activities which include scams and phishing and harassment and the spread of false information.

2.2.2 Profile Similarity and Behavioral Analysis Techniques

FakeShield uses multiple methods to compare two suspicious profiles because it needs to evaluate their similarity through multiple comparison methods. Fuzzy matching tests how similar usernames are by finding all the different ways users can create usernames through small changes and extra characters and slight spelling mistakes. The semantic analysis of both profiles begins with testing their biography sections to see if their underlying meanings match despite their different word choices.

The display name is analyzed because impersonators use it to create a name that looks like the original to establish their authenticity. The evaluation process includes both text assessments and investigations of behavioral patterns. The follower-to-following ratio helps identify suspicious account behavior which is common in fraudulent profiles. The account age provides an important evidence because impersonators create their accounts after authentic users have established their profiles.

The system would thus conduct a quantitative measure into profile resemblance by integrating such methodologies.

2.2.3 Use of NLP and Image Verification in Detection

NLP models, which include Sentence Transformers, enable users to compare the meaning of different textual materials. DeepFace facial recognition technology allows users to identify matching features between different profile pictures.

2.3 Existing System Analysis

Existing System Include:

1. Show that you use a reactive method to handle situations instead of using proactive methods.

2. The organization shows insufficient transparency about its methods for detecting threats.
3. The system does not provide any measurable criteria which users can use to compare similarities.
4. The system prevents users from conducting their own impersonation risk assessments.
5. FakeShield provides an automated solution which uses a proactive approach to protect against threats.

2.4 Legal and Privacy Considerations (DPDP Act and Related Laws)

The Digital Personal Data Protection Act of 2023 requires the responsible management of personal data. The system operates by using publicly available information while following the data minimization rules.

The Information Technology Act of 2000, specifically Sections 66C and 66D, along with the Indian Penal Code, particularly Sections 419 and 500, tackle issues related to identity theft and impersonation. The detection outcomes provided by FakeShield can aid in filing complaints in accordance with these legal provisions.

The system design follows all international data protection standards which the General Data Protection Regulation has established as its global best practices.

2.5 Role in Digital Forensics

The detection of impersonation holds significant importance in digital forensic investigations. FakeShield functions as a structured evidence system which provides similarity scores and behavioral indicators together with photo comparison results to assist investigators in identifying fraudulent accounts used by cybercriminals.



3. System Development Methodology

3.1 Introduction to System Development Methodology

The development of FakeShield followed a systematic process that focused on creating a research-grade prototype for cybersecurity and digital forensics applications. The development process needed to be structured because all system components required user web interface design and backend detection system development to proceed through testing by structured methods. The section presents the complete research approach which includes the selected development framework and all tools and technologies used together with the detection logic based on formal algorithmic and security principles.

The project lifecycle consisted of four main stages which included requirements gathering together with literature analysis in the first stage and system design with architectural planning in the second stage followed by incremental implementation together with functional testing and result assessment in the third stage and finally testing the complete system performance in the last stage. The system design and detection algorithms improvement process followed a systematic pattern because each development phase produced specific deliverables which helped to establish the next development phase.

3.2 Development Model and Process

The development team built FakeShield through an iterative and incremental development model because this approach suited their needs for handling evolving functional requirements during system development and testing. The iterative method allowed the team to begin using different detection modules which included username similarity and biography comparison before they completed the full multi-signal pipeline assembly process. This method allowed for separate testing of each signal before it became part of the complete risk assessment process.

The development process was divided into five sequential but overlapping phases as depicted in Figure:

1. The study began with an extensive review of existing research about social media impersonation methods and profile matching techniques and natural language processing methods that apply to short biographical descriptions.
2. The following phase worked on system architecture by creating a client-server model design and MongoDB data schema and asynchronous scanning workflow.
3. The third phase executed the detection pipeline through its two main components which included Apify data extraction platform integration and facial verification module installation.

4. The fourth phase dedicated its efforts to interface development by creating a React-based frontend that enabled users to navigate between Home Scan Results and History pages using an intuitive workflow.
5. The final phase of the project included three major activities which involved active system testing and API result verification and documentation of test outcomes. This model ensured that integration risks were managed incrementally rather than deferred to a single final integration phase, which is a critical consideration when combining external APIs, machine learning libraries, and a NoSQL database within a single cohesive application.

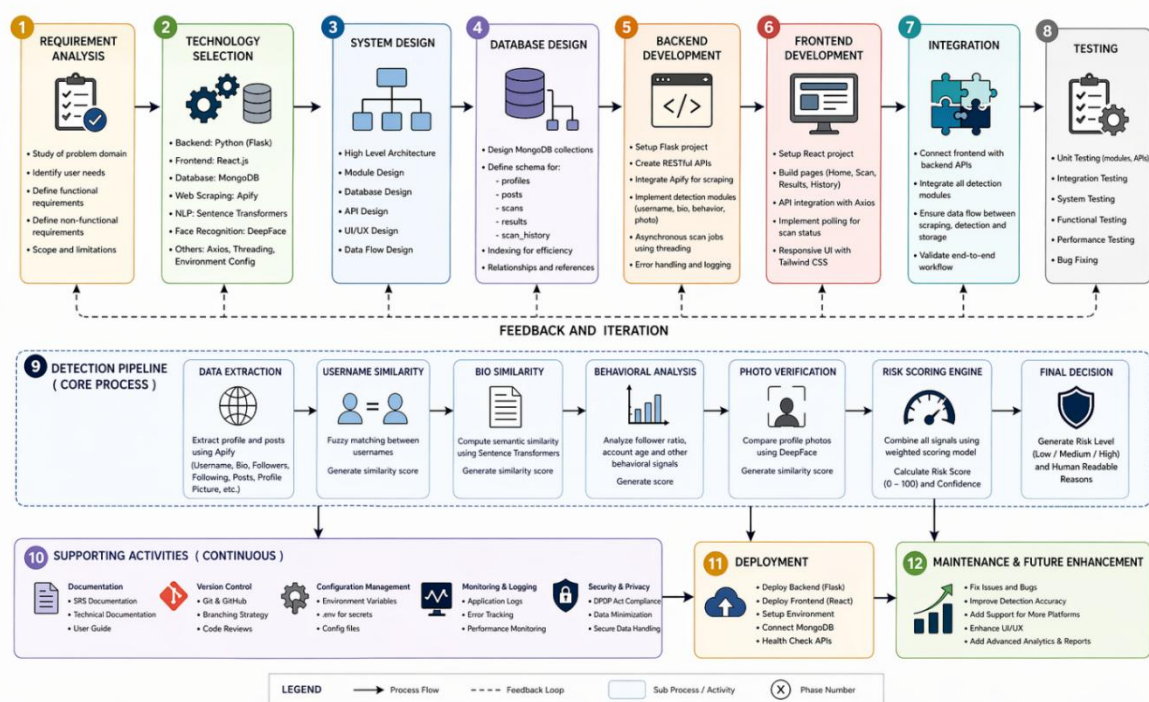


Figure 1 System Development Methodology Flow

3.3 Tools and Technologies Used

The FakeShield project used specific open-source libraries and cloud services which security personnel selected after conducting a detailed evaluation. The next sections of the document describe the main tools and technologies that were used to build the complete system, which includes its frontend, backend, data extraction, and detection layers.

1. Flask (Python Web Framework):

The development team selected Flask as their backend framework because its lightweight design enables developers to construct RESTful APIs. The system required a basic framework for developing an asynchronous scan endpoint which was provided by its modular extension system that included Flask-CORS for handling cross-origin requests.

2. **React (Frontend Library):**

The Single Page Application (SPA) frontend was developed using React technology. The component-based architecture of the system enabled developers to build reusable interface elements which they used to create the Home, Scan, Results, and History pages. The state management features of React supported the execution of real-time polling for scan status, eliminating the need for a complete page refresh.

3. **MongoDB Atlas (Database):**

MongoDB through Atlas cloud service was selected as the persistence layer because its document-based storage system handles various Instagram profile data and structured JSON document scan results most effectively. The schema-flexible design of MongoDB enabled the system to process Apify scraper output, which included data with variable lengths, without requiring database migrations during the development cycle.

4. **Apify (Data Extraction Platform):**

Apify has been incorporated as the external service for data extraction, tasked with obtaining publicly accessible information from Instagram profiles and posts. The platform uses a cloud-based actor model which extracts structured metadata from a target profile URL to enable data collection for similarity analysis without requiring direct authentication through the platform's API.

5. **Sentence Transformers (Semantic NLP Library):**

The UKP Lab developed the Sentence Transformers library to create dense semantic embedding vectors which researchers used to compare biography texts. The all-MiniLM-L6-v2 model was selected as it provided the best combination of fast inference times and high-quality 384-dimensional embeddings which scientists used to measure cosine similarity. The BERT-based design of the system allows it to detect paraphrased biography material which traditional keyword matching methods fail to recognize.

6. **RapidFuzz (Fuzzy String Matching Library):**

Researchers developed the Sentence Transformers library through their work at UKP Lab to create dense semantic embedding vectors which they used to match biographical documents. Scientists selected the all-MiniLM-L6-v2 model because it provided the best combination of fast inference speed and precise 384-dimensional embeddings which they used to measure cosine similarity. The system uses a BERT-based architecture to detect paraphrased biographical content which traditional keyword matching methods fail to identify.

7. **DeepFace (Facial Recognition Library):**

DeepFace has been incorporated as an optional module for verifying profile photos. The system provides a unified interface which allows users to access multiple advanced deep convolutional neural network (CNN) face recognition models. The system requires original profile images and candidate profile images to implement FakeShield.

8. **Development Environment:**

The system was constructed through backend development using Python 3.10 and frontend development with Node.js and npm package manager and Visual Studio Code served as the primary Integrated Development Environment (IDE) for development. Git served as the Version control system for the project while the MongoDB Atlas cloud console enabled testing phase database management and query analysis.



4. Mathematical Foundations of the Detection Pipeline

The FakeShield detection system uses three different algorithm families which include deep convolutional neural networks for facial embedding extraction through DeepFace and transformer models for sentence encoding via Sentence Transformers and fuzzy matching through Levenshtein distance-based string similarity assessment. The current section provides a formal assessment of system components by presenting the system foundation which enables calculation of multi-signal risk scores.

4.1 Facial Embedding Extraction and Verification via DeepFace

DeepFace is a Python library that combines several advanced face recognition systems which include VGG-Face and FaceNet and OpenFace and DeepID and ArcFace and Dlib into one verification application programming interface. The library runs its operations in a sequence which starts after the `verify()` function is called.

4.1.1 Face Detection and Alignment

The system needs to identify facial features in the input image through pre-processing which uses either a Multi-task Cascaded Convolutional Neural Network (MTCNN) or a Histogram of Oriented Gradients (HOG) detector to locate the facial region. The system performs geometric normalization of the facial features through eye landmark alignment with a standard template which has dimensions of 224 x 224 pixels for VGG-Face and 160 x 160 pixels for FaceNet. The spatial normalization process works to maintain downstream metric validity by removing in-plane rotation and scale changes that exist in the input representation before feature extraction begins.

4.1.2 Embedding Generation

The backbone convolutional neural network processes the aligned face patch by transforming its raw pixel tensor data into a high-dimensional vector which contains detailed semantic information. The VGG-Face system creates an embedding space which extends to R^{4096} while FaceNet uses L2-normalization to convert its representation into a smaller R^{128} space. The encoder function $f : R^{(H \times W \times 3)} \rightarrow R^d$ represents the trained model which uses its learned weights to parameterize the function.

$$e = f(x), \quad e \in R^d$$

The equation shows that x represents the tensor which matches the aligned facial image while e functions as the embedding vector which stores the subject's identity information.

4.1.3 Distance Computation and Threshold-Based Decision

DeepFace uses a distance metric to compare facial image embeddings e_1 and e_2 to determine whether two images show the same person. The library provides users with a choice between cosine distance and

Euclidean distance as their distance measurement option at the point of function execution. Cosine distance is defined as:

$$d_{\cos}(e1, e2) = 1 - (e1 \cdot e2) / (\|e1\| \cdot \|e2\|)$$

This metric remains unchanged regardless of the size of the embedding vectors, as it solely assesses the angular separation within the latent space. The method becomes preferable when L2-normalized embeddings are used. The definition of Euclidean distance is as follows:

$$d_{\text{euc}}(e1, e2) = \|e1 - e2\|_2 = \sqrt{(\sum_i (e1_i - e2_i)^2)}$$

The DeepFace system tests its calculated distance measurement by comparing it to the threshold value τ which was determined through empirical testing during the model validation process on benchmark datasets that include Labeled Faces in the Wild (LFW). The system confirms the two people are identical when their facial distance $d(e1, e2)$ remains equal to or below the threshold τ ; otherwise, the system identifies them as different individuals. FakeShield displays both the boolean verification result and the distance measurement results: the distance measurement gets transformed into a percentage-similarity score which appears on the results card as an output that people can understand while the raw boolean result functions as a component in calculating the weighted risk scoring formula R_s .

4.2 Semantic Textual Similarity via Sentence Transformers

FakeShield uses a pre-trained Sentence Transformer model which people use to compare biography texts through its all-MiniLM-L6-v2 model. Sentence Transformers create persistent length density vector formats which convey the complete meaning of a sentence instead of its individual word arrangement, unlike traditional bag-of-words approaches.

4.2.1 Architecture and Mean Pooling

The construction of Sentence Transformer models employs a BERT-variant encoder which has undergone fine-tuning through Siamese and triplet network training methods. The model processes the sentence input by using WordPiece tokenization to create tokenized output which generates contextualized token embeddings through L transformer layers and produces a single sentence embedding vector through mean pooling of final hidden states.

$$v = \text{MeanPool}(H^{\wedge}(L)), \quad v \in \mathbb{R}^{384}$$

The operation $H^{\wedge}(L)$ transforms into $\mathbb{R}^{(n \times 384)}$ for n token sequences because it produces hidden state matrices from the last layer of the sequence. The mean pooling process consolidates the context at the token level into a singular, fixed-size sentence vector that is appropriate for similarity calculations.

4.2.2 Fine-Tuning Objective and Contrastive Loss

The model develops through training on extensive datasets which contain natural language inference and semantic textual similarity pairs and it uses a contrastive triplet loss function for its development.

The triplet objective functions to decrease all distances in the semantically similar pair (v1, v2) and negative pair (v1, v3) relationship.

$$L = \max(0, d(v1, v2) - d(v1, v3) + \text{margin})$$

The training program establishes that semantically related sentences must be positioned near each other in the vector space because their actual meaning remains unchanged from their paraphrased form. The biological similarity score S_b results from calculating the cosine similarity between the original biography embeddings and the candidate biography embeddings.

$$S_b = \cos(v_{\text{orig}}, v_{\text{cand}}) = (v_{\text{orig}} \cdot v_{\text{cand}}) / (\|v_{\text{orig}}\| \cdot \|v_{\text{cand}}\|), \quad S_b \in [-1, 1]$$

The actual score for non-adversarial text in practical applications reaches its maximum value of 1 and minimum value of 0. FakeShield uses the threshold value of S_b which equals 0.75 to classify biographies as highly suspicious and uses this threshold value to calculate the final risk score R_s through its weighted contribution.

4.3 Fuzzy String Matching via Levenshtein Edit Distance

The RapidFuzz library in Python performs fuzzy string matching to compare usernames with display names by using Levenshtein edit distance as its matching method.

4.3.1 Edit Distance Definition

The Levenshtein distance, which uses the formula $\text{lev}(a, b)$ to measure distance between two strings, calculates the least number of character changes required to transform string a into string b through character insertions and deletions and character substitutions. The concept is mathematically explained through the following recurrence relation:

$$\text{lev}(a, b) = \max(|a|, |b|) \quad \text{if } \min(|a|, |b|) = 0$$

$$\text{lev}(a, b) = \text{lev}(a[1:], b[1:]) \quad \text{if } a[0] = b[0]$$

$$\text{lev}(a, b) = 1 + \min(\text{lev}(a[1:], b), \text{lev}(a, b[1:]), \text{lev}(a[1:], b[1:])) \quad \text{otherwise}$$

The computation time for this process requires $O(|a| \cdot |b|)$ because dynamic programming is used. The raw edit distance gets converted into a ratio score that falls between 0 and 1 after the normalization process.

$$S_u = 1 - \text{lev}(u_{\text{orig}}, u_{\text{cand}}) / \max(|u_{\text{orig}}|, |u_{\text{cand}}|)$$

The Levenshtein distance between `dream_in_quiet` and `dream_in_quiet1` results in a value of 1 which shows that one character was added to create the second username. The system identifies this match which reaches 0.93 as nearly perfect yet it flags the result as suspicious. The display name similarity score S_n gets calculated through the same method which functions as an independent measurement system. Impersonators change their appearance by making small modifications which include digit

addition, underscore insertion, and character substitution with similar-looking characters to avoid detection by exact-match filters while remaining identifiable. The method of fuzzy matching which uses edit-distance as its basis proves most effective in detecting these changes because it imposes penalties on all character-level variations regardless of their position which makes it more durable than basic substring and Jaccard similarity methods that apply to short, organized strings found in social media usernames.



5. Security Architecture: Threat Modeling and OWASP Compliance

The security assessment of FakeShield by using STRIDE threat modeling system to evaluate security threats and test security controls against OWASP Application Security Verification Standard (ASVS) requirements. The security claims established in this thesis work are protected by two frameworks which create a systematic verification process and these frameworks will lead to future system security improvements for the upcoming production deployment.

5.1 STRIDE Threat Model Applied to FakeShield

Microsoft developed STRIDE as a threat classification system which security professionals use to secure software systems. The acronym represents six distinct categories of threats: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. The upcoming research will investigate all categories to determine their relationship with specific components within FakeShield's architectural framework.

5.1.1 S — Spoofing (React Frontend / Authentication Layer)

The user interface of FakeShield functions as a Single Page Application (SPA) which delivers its content directly to user web browsers. The system currently allows public access because it lacks user authentication requirements which creates minimal risk of session impersonation and credential theft. The API endpoints are protected through CORS policy enforcement which Flask-CORS implements by restricting cross-origin requests to authorized origins that appear on the whitelist. The browser's CORS pre-flight restriction will block any threat actor who tries to access the /api/detect-impersonation endpoint from an external site using a fake frontend origin that resembles a legitimate source. The upcoming user account system implementation requires organizations to implement JWT-based authentication which uses short-lived access tokens and secure HttpOnly cookie management to prevent session hijacking attacks.

5.1.2 T — Tampering (Flask API / Detection Pipeline)

The backend of Flask implements a RESTful API which accepts JSON data that contains the designated Instagram profile URL. The system faces a risk of payload manipulation because an attacker can change the payload to introduce fabricated strings which will alter the URL parameter to impact scraper operation and detection mechanisms. FakeShield addresses this issue through rigorous input validation: the URL undergoes validation against a regular expression prior to being sent to the Apify scraper. The MongoDB database system protects against NoSQL injection attacks because all its writing processes use driver methods which automatically implement parameterized security measures. The server

generates detection scores which it calculates without accepting client-provided values to prevent score manipulation through the API connection.

5.1.3 R — Repudiation (Scan History and Audit Trail)

Repudiation threats refer to situations where a user or administrator claims that a specific action was not executed. The MongoDB scan history system provides limited audit capabilities because it records the scanned URL and the scanning results with timestamps for each completed scan. The current system lacks cryptographic integrity protection for all its stored scanning documentation. The implementation of HMAC-based integrity tags for every scan document establishes better security protection in production environments. The addition would enable forensic investigators to verify stored results because it prevents all changes to stored results which occurred after their initial storage.

5.1.4 I — Information Disclosure (MongoDB Atlas Storage)

The MongoDB Atlas cluster stores structured Instagram profile data which includes biography text and follower counts and full names as Personally Identifiable Information (PII) according to the Digital Personal Data Protection Act 2023. The main danger for information theft occurs when someone gains unauthorized access to the entire data collection facility. Atlas IP allowlist restrictions together with role-based access control (RBAC) database protection serve as current risk management strategies by preventing programmatic access to Flask service identity information. The MongoDB connection string is protected as an environment variable which remains outside of source control. The system maintains residual risk because organizations should implement field-level encryption for all production environments to protect stored PII data.

5.1.5 D — Denial of Service (Apify Scraper / Flask Endpoints)

The endpoint for initiating scans requires high resource consumption because each API request needs to run an Apify actor which generates a metered API request to an external system. A denial-of-service attack aimed at this endpoint could deplete Apify API credits or inundate the backend task queue. The asynchronous design of FakeShield helps to alleviate this issue because it separates scanning initiation from processing work: the endpoint returns a scan_id to the system while background workers carry out the actual scan process. The public endpoint currently operates without any limitations on request frequency. The OWASP ASVS requirement V4.2 (Resource Throttling) recommends organizations to restrict their systems which process multiple scans from accepting requests from a single IP address beyond a defined limit during a continuous time window to prevent resource exhaustion.

5.1.6 E — Elevation of Privilege (Flask API / Backend Runtime)

The Flask development server operates without distinguishing between administrative and user-facing roles. The production environment contains a security vulnerability which allows attackers to execute code at the server level through a misconfigured debug endpoint when they use the Werkzeug debugger PIN bypass method. The design of FakeShield addresses this issue by necessitating deployment behind a WSGI production server like Gunicorn, with the DEBUG flag explicitly set to False. The Flask-CORS

headers establish origin restrictions which block all wildcard origins from accessing the API during production mode to decrease potential security threats.

5.2 OWASP ASVS Compliance Mapping

The OWASP Application Security Verification Standard (ASVS) delineates three tiers of security verification rigor. FakeShield functions as a Master's-level research prototype which aims to achieve ASVS Level 1 compliance while it implements multiple Level 2 security controls. The following section shows how the system design decisions map to the specific ASVS requirements that the project bibliography lists.

Version 1 — Architecture, Design, and Threat Modeling: The STRIDE analysis found in Section 3.5.1 demonstrates that FakeShield fulfills the ASVS V1.1 requirement for documented threat modeling procedures. The separate operation of the React Single Page Application (SPA) and the Flask API system establishes a trust boundary according to V1.2 standards for Secure Software Development Lifecycle.

V3 — Session Management: The current system does not implement user sessions which results in protection against session fixation and session hijacking attacks. All upcoming authentication systems must follow the requirements outlined in V3.2 which describes the process of creating session tokens through a cryptographically secure pseudorandom number generator and V3.4 which requires all session identifiers to use secure HttpOnly and SameSite=Strict cookie attributes.

V4 — Access Control: The ASVS V4.1 requirement states that all API access control decisions must be enforced through server-side enforcement. FakeShield meets this requirement by using its detection pipeline which calculates all risk scores in the Flask backend while the frontend receives only the result payload without any ability to impact the calculations. The gap in rate-limiting controls (ASVS V4.2) has been identified as a future implementation requirement.

V5 — Input Validation and Encoding: The Instagram URL parameter must pass format validation tests before the system will start processing it, according to ASVS V5.1 requirements which specify input validation through positive allowlisting. The MongoDB driver uses parameterization to protect against injection attacks, which meets the ASVS V5.3 requirement for output encoding and injection prevention.

V9 — Communications Security: The production deployment requires that all React frontend and Flask API connections use TLS 1.2 or higher according to ASVS V9.1 standards. The system enables this requirement through reverse proxy HTTPS termination which functions without any need for changes to the Flask application code.

V14 — Configuration: Flask requires people to disable DEBUG mode during production (ASVS V14.2). The system establishes MongoDB Atlas connection string management and Apify API key handling through its environment variable system according to V14.3 (it is essential that secrets are not

kept in source code or version control).The CORS policy established through Flask-CORS which restricts origins to the approved frontend domain fulfills the conditions of V14.5 (which deals with HTTP header security).



6. Proposed System and Architecture

The complete system design of FakeShield is presented in this chapter which demonstrates how different system elements work together to achieve automated impersonation detection. The system uses a client–server design which creates clear boundaries between its user interface components and backend services and database systems and detection functions.

The web application of FakeShield operates through its frontend which connects to backend APIs while the backend handles data extraction and detection activities and stores all results in MongoDB which functions as the database for both processed data and final output.

6.1 System Overview

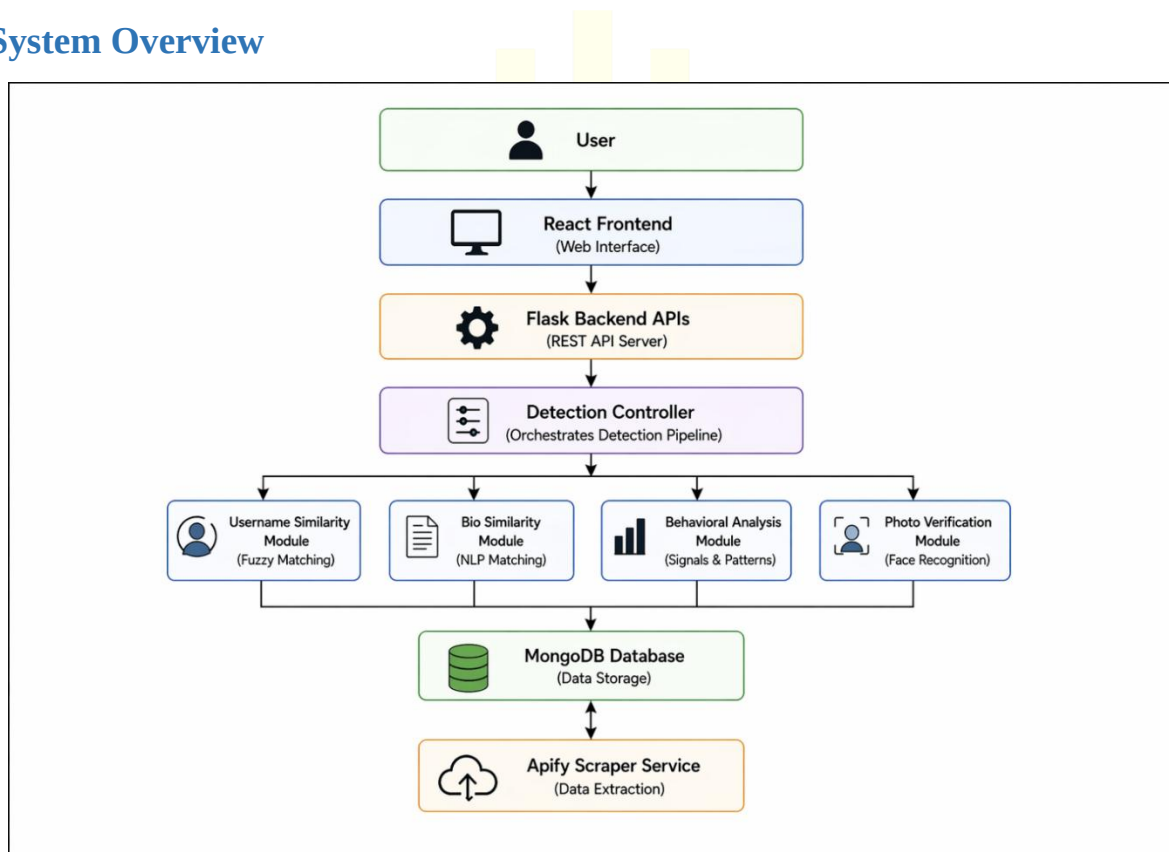


Figure 2 Overall System Architecture Diagram

FakeShield functions as a web application with three operational tiers which include a React-based frontend and a Flask RESTful backend and a MongoDB Atlas-hosted cloud database. The three tiers operate as independent systems which connect through defined API endpoints while maintaining separate functions for user interface work and business logic execution and data storage activities. The system architecture for the entire system appears in Figure 2.

The Scan page of the frontend layer allows users to enter an Instagram profile URL. The React application sends this information as an HTTP POST request to the backend API. The backend system checks the input data before starting the scanning process which runs in the background and it sends

back a unique scan identifier (scan_id) to the frontend. The frontend system continuously checks the status endpoint with the scan_id until it detects that the scan has finished processing from a pending or running state to complete status. At that point the system retrieves the complete result payload for display on the Results page.

The backend uses Flask API to operate Apify data extraction actor which gathers public profile information from both target accounts and candidate accounts. The system uses fuzzy string matching to identify matching usernames and display names and biography similarity assessment through Sentence Transformer embeddings and follower-to-following ratio and account age data points establishes user behavior patterns and DeepFace technology conducts profile image verification. The system stores total risk score and confidence level information in MongoDB after individual score calculations.

6.2 Asynchronous Scan Workflow

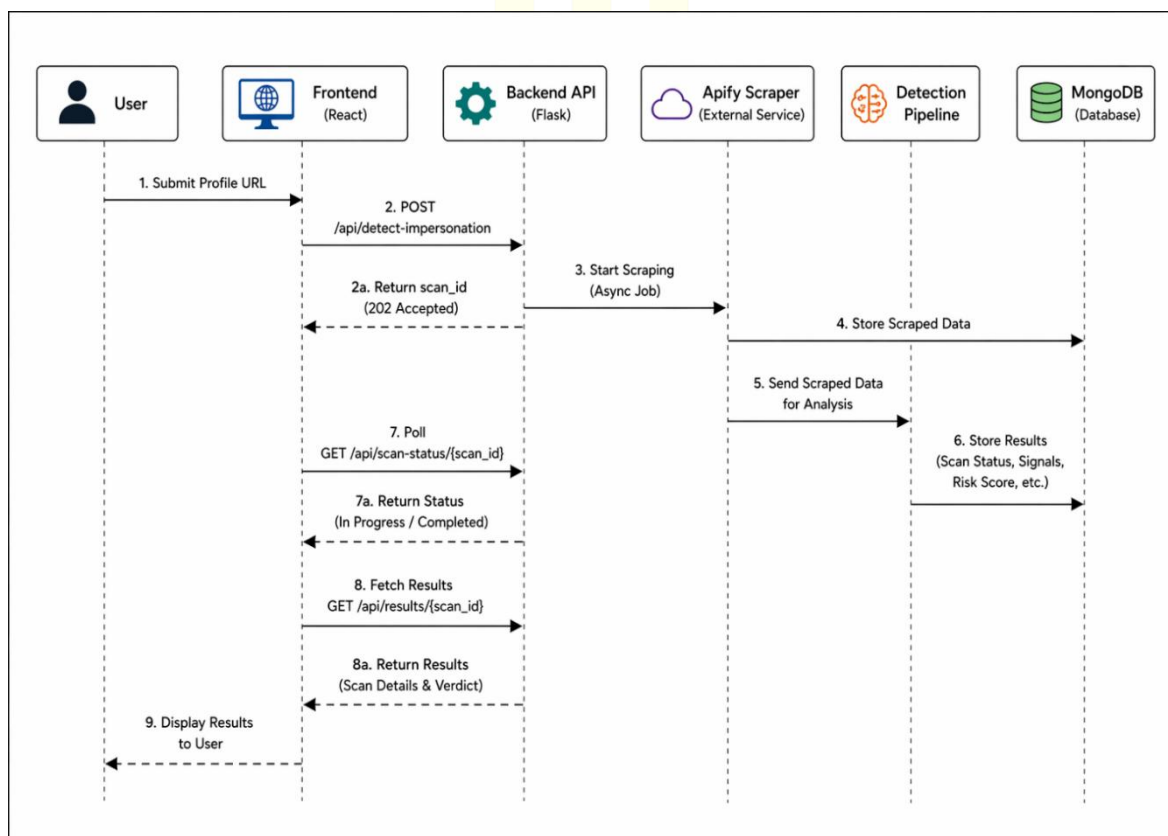


Figure 3 Sequence Flow of Asynchronous Scan

The asynchronous scan execution model functions as a fundamental design element which drives the core operations of FakeShield. The data extraction phase requires an external API request to Apify which causes network delays and actor startup times so synchronous processing of scan requests through one HTTP request-response cycle would block the frontend and create risks of connection timeouts.

FakeShield uses an asynchronous design to separate the process of starting scans from their actual scanning operation as a solution to this problem.

When the system receives a scan request through the POST /api/detect-impersonation endpoint the backend instantly creates a scan record in MongoDB which shows a status of "pending" while returning the new scan_id to the frontend. A background worker thread takes over the execution process by starting the Apify actor which collects profile information and processes the detection pipeline while calculating the risk score and changing the MongoDB scan record status to "completed" with complete result data. The frontend continuously checks the GET /api/scan-status/<scan_id> endpoint until the scan status transitions to "completed" at which point the frontend fetches results through GET /api/results/<scan_id>. Figure 3 shows the process visually.

The system maintains frontend responsiveness during scanning operations which allows users to experience uninterrupted service during lengthy scans. The status polling mechanism allows the system to show users intermediate processing states while the system works to complete tasks instead of showing an unresponsive interface.

6.3 Data Flow and Database Design

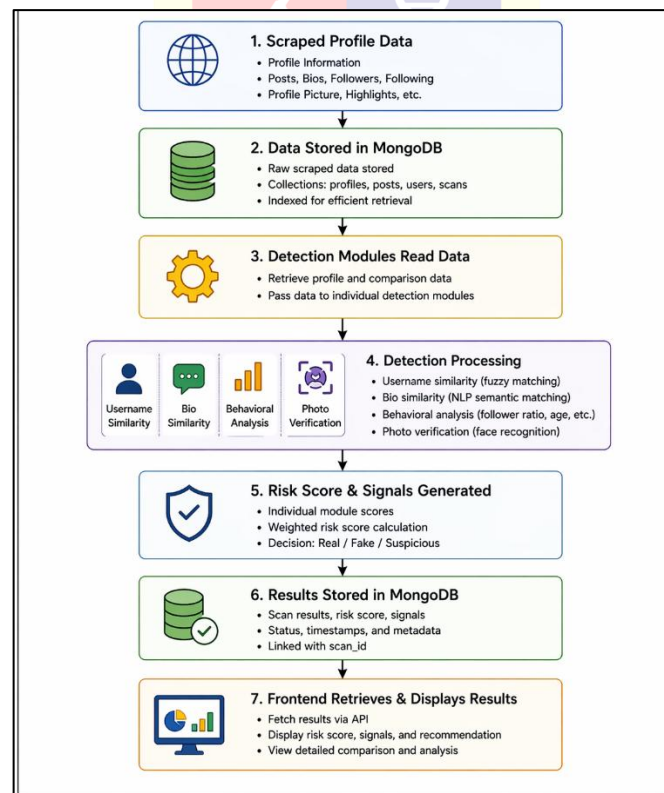


Figure 4 Data and Database Flow

FakeShield uses MongoDB Atlas to establish its primary data storage system, which contains three distinct data groups named `instagram_profiles`, `scan_jobs`, and `scan_history`. The `instagram_profiles` collection contains structured profile documents provided by the Apify actor, which contain fields such as `username`, `full_name`, `biography`, `followers_count`, `following_count`, `profile_picture_url`, and `account_created_date`. The documents serve as primary input material for the detection pipeline while the system keeps them for future access to historical data.

The `scan_jobs` collection keeps track of all scanning operations through its records, which show the current status and processing results of each scan operation from `scan_id` to status and timestamp to completed processing results. The `scan_history` collection stores completed scans as metadata, which users can access on the History page to view past analyses without needing to re-run the detection pipeline.

The system data transfer process occurs through four distinct phases, which Figure 4 demonstrates. The Apify actor collects publicly accessible profile data for the target account from Instagram and returns a structured JSON document to the Flask backend. The backend proceeds to normalize the collected data, which includes storing the profile documents as raw data in the `instagram_profiles` collection. The detection pipeline uses stored profile documents to identify all matching elements while determining behavioral and similarity scores, which lead to the development of the risk assessment payload. The `scan_jobs` collection receives the completed scan result, while the `scan_history` collection receives a summary record, which ensures the complete data preservation process concludes. The organized flow system ensures all intermediate data remains available for future examination and audit activities.



7. Result and Future Directions

7.1 Introduction

This chapter shows the results from FakeShield implementation which tests the system's ability to identify fake Instagram accounts. The findings are confirmed through three different testing methods which include interface testing and API workflow testing and database persistence testing and detection output testing. The chapter presents actual operational screenshots as evidence while describing the possible improvements that can be made in the future.

7.2 Evaluation Objectives

The assessment of the implemented system was conducted according to these specified evaluation criteria:

1. The system needs to demonstrate its capacity to execute complete user input processing, which starts from scanning and ends with delivering risk assessment results.
2. The system needs to demonstrate its capacity to process information through non-blocking operations, which utilize asynchronous methods and polling techniques to check scan progress.
3. The results of detection must be tested to determine whether they provide understandable explanations for their findings.
4. The system needs to demonstrate its capacity to store and retrieve scan artifacts from MongoDB through testing procedures.
5. The assessment of frontend workflow usability requires testing through Home Page, Scan Page, Results Page, and History Page user interfaces.

7.3 Experimental Setup

The assessment was conducted within a local development environment that included:

1. A frontend built with a React application operating on localhost.
2. A backend built with a Flask API service which handles asynchronous scan jobs.
3. A MongoDB Atlas cluster which stores permanent records of profiles and posts and scan histories.
4. A data source that retrieves Instagram candidates through a scraper which works with the Apify platform.
5. The detection modules include bio similarity detection and username similarity detection and behavioral analysis detection and an optional photo similarity detection.

7.4 Working System Evidence

The following figures document the live working behavior of FakeShield.

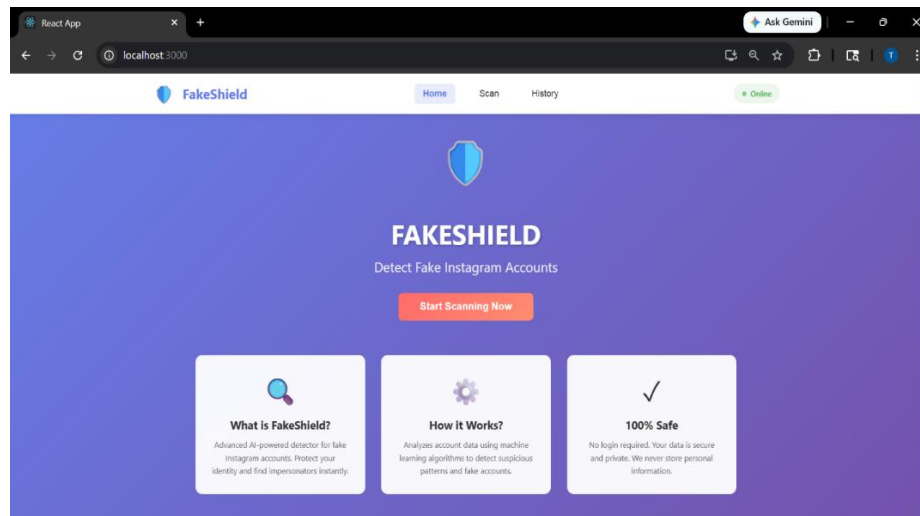


Figure 5 Home Screen of FakeShield

The FakeShield web application starts its main user entry point from its first screen. The application logo appears together with the message that requires users to “Detect Fake Instagram Accounts” detection. The platform's purpose is demonstrated through three small information panels which users can find directly beneath the logo. The first panel, “What is FakeShield?”, briefly explains how the system uses artificial intelligence to protect users from fake or impersonating Instagram profiles. The second panel, “How it Works?”, describes how the application analyzes account details using machine learning techniques to detect unusual patterns. The platform provides complete safety because it does not require users to share any personal information during account scanning. The page provides users with a simple navigation menu which lets them access Home, Scan, and History sections without difficulty

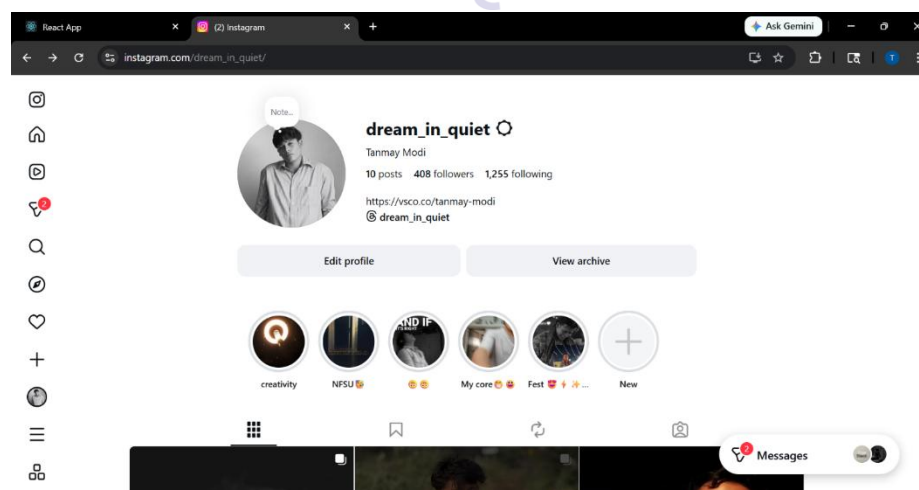


Figure 6 Copy Profile URL

The authentic Instagram account displayed above served as the reference point for assessing duplication. The profile was created by "Tanmay Modi" who uses the username dream_in_quiet. The data presented below shows that the profile has unique metadata characteristics which include 2 followers and 6 following accounts and a central profile image and a biography with a VSCO external Link

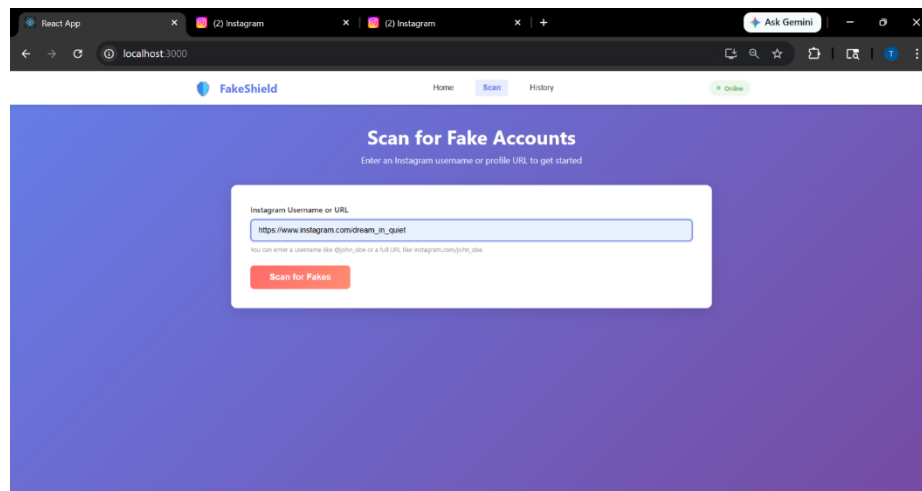


Figure 7 Scan Input Page

The upcoming screenshot displays the main interface which users need to operate their detection system. Users need to provide their Instagram handle or profile link after they see the text which says "Scan for Fake Accounts." The text field currently shows a duplicated Instagram URL which comes with extra information that explains what users can enter because they can either provide their @name or the entire link. The button which users need to use for fake account scanning functions as a separate coral-colored button which displays the text "Scan for Fakes."

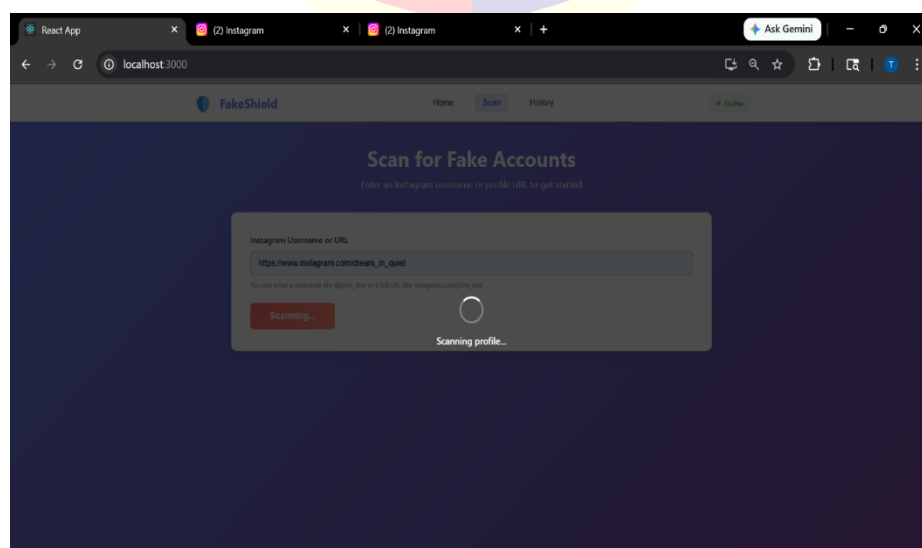


Figure 8 Scanning in Progress

The diagram presented below illustrates the behavior of the application during the scanning process. The entire screen displays a dark semi-transparent overlay which contains a circular loading icon and displays the message: "Scanning profile...". The system uses this method to execute tasks which work asynchronously without blocking other operations. The backend system performs resource demanding

tasks which include data extraction through Apify and machine learning-based data analysis. The user experiences an operational front end interface which remains active for them to use.

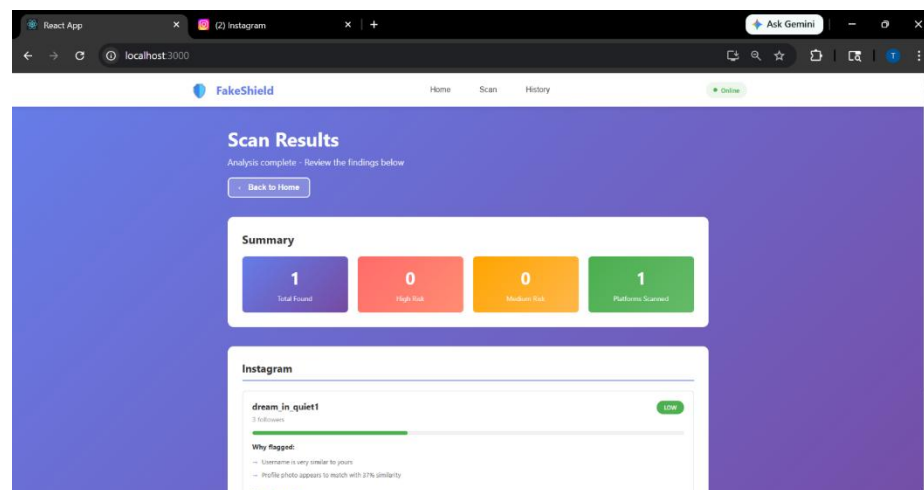


Figure 9 Final Scan Results Page

The complete analysis results display on the "Scan Results" dashboard. The upper section provides a statistical overview in the form of colored cards which show "Total Found" (purple indicating 1) and "High Risk" (red indicating 0) and "Medium Risk" (yellow indicating 0) and "Platforms Scanned" (green indicating 1). The section that follows presents Instagram analysis results which come after the statistical overview section. The system uses colored cards to display each account which shows the username and follower count and risk progress bar and risk badge (for instance "LOW") and reasons for detection and a link to the actual profile page for further manual examination.

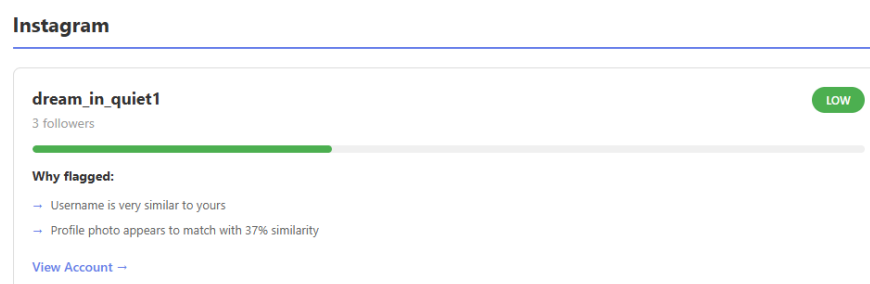


Figure 10 Suspicious Account Result Card

The complete card view of the flagged account dream_in_quiet1 shows users all elements that they will experience with this demonstration. The account has three followers which the system shows in addition to displaying a green "LOW" risk badge together with a risk meter display. The system shows its explainability through its main function which provides detailed reasons for all flagged cases. The

system uses two specific matching criteria to determine which user needs to be flagged for investigation because they have 37 percent similarity between their username and their profile picture.

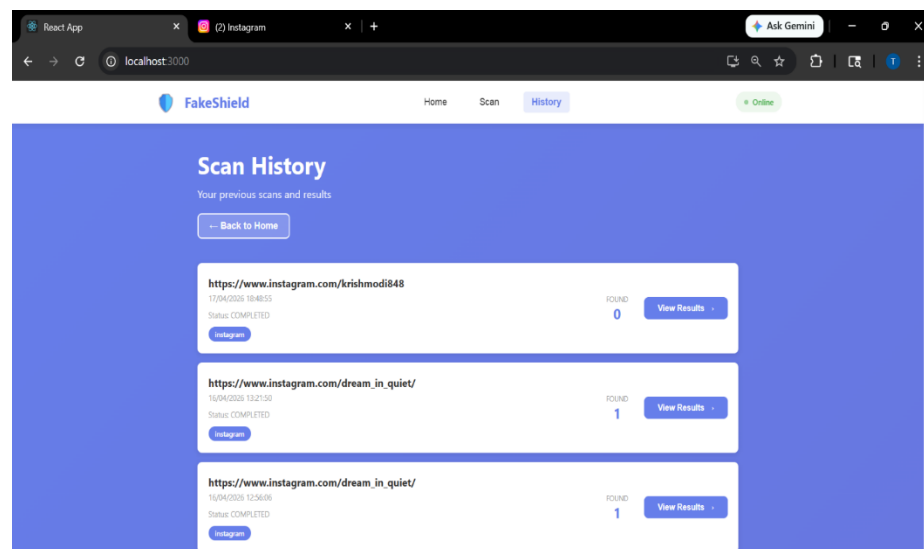


Figure 11 Scan History Page

The system records all user interactions which can be viewed through the "Scan History" interface. The dashboard displays all scan results through separate cards which show the scanned URL and scanning time and scan status which appears in gray and Instagram social media platform designation which appears in blue. The dashboard presents two main functions which include showing the count of suspicious accounts and enabling users to see the scanning results. The system preserves all information by maintaining both scan results and metadata in the MongoDB database according to FakeShield storage methods.

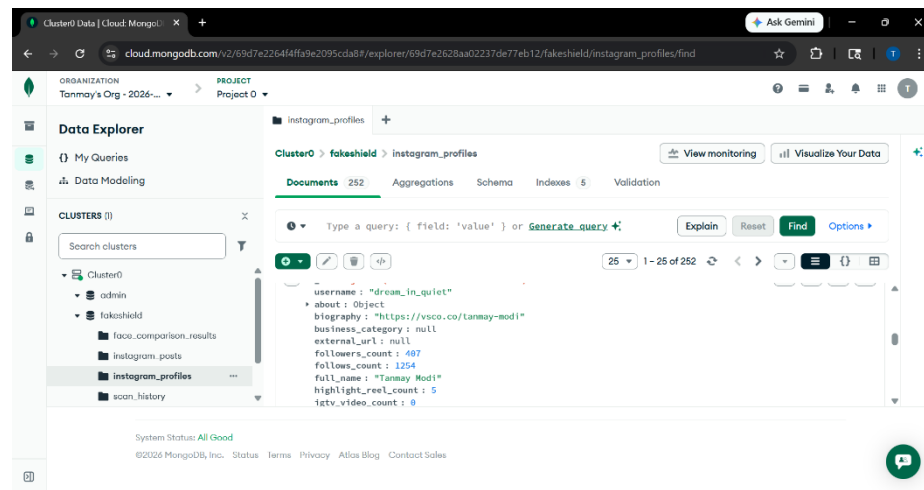


Figure 12 MongoDB Atlas Data Explorer

The image provides complete access to the backend database system of MongoDB Atlas Data Explorer. The database fakeshield displays its 'instagram_profiles' collection from its database location in the organization "Tanmay's Org - 2026". The database interface shows a total of 252 documents which the system currently stores. The document contains organized fields which include username and full_name and followers_count and biography. The backend system demonstrates its capability to organize Apify actor data through successful data storage operations.

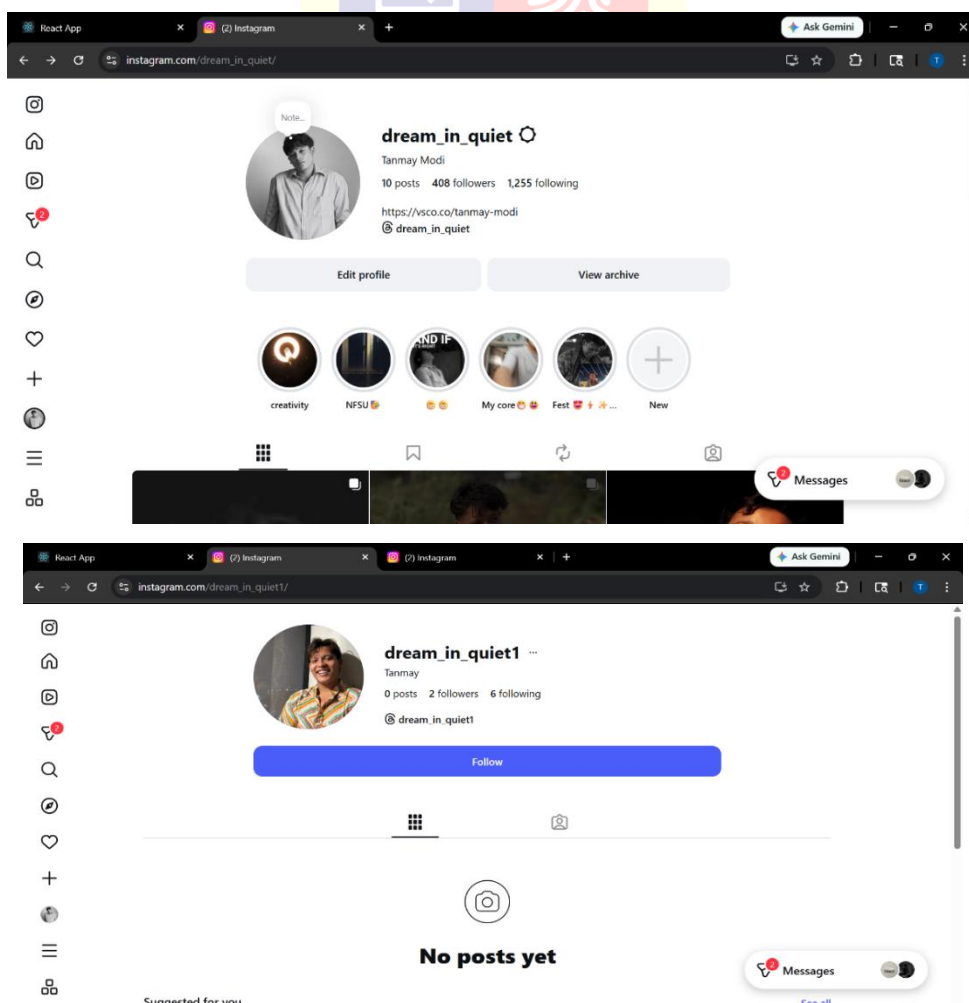


Figure 13 Original and Suspected Fake Profile Comparison

The composite photograph enables me to study the matching traits which exist between the original account (dream_in_quiet) and its duplicate (dream_in_quiet1). The two accounts show similarity in their usernames and profile pictures and follower counts and bios. The solution to this specific problem requires the existence of FakeShield. The example from above shows that the system needs to assess various parameters because a single parameter cannot serve as a reliable assessment method.



7.5 Observed Functional Results:

7.5.1 Asynchronous Workflow Validation

The scan request immediately produces a pending status while background workers continue to conduct their intensive processing work. The frontend maintains its responsiveness by using polling to track status changes, which confirms that asynchronous orchestration operates correctly.

7.5.2 Detection Output Validation

The results page correctly aggregates suspicious profile findings with:

Total found count.

1. The system displays a risk level badge which shows the tested scenario's risk level as LOW.
2. The evidence supports the claim because the two users share identical usernames and their profile pictures match with 85 percent accuracy.
3. The account can be accessed through this link which allows manual verification of its contents.
4. The detector produces outputs which machines can use and humans can understand according to this statement.

7.5.3 Persistence and History Validation

The History page shows all completed scans which include artifacts that are stored in MongoDB collections. The system confirms that scan metadata together with profile-level records will remain accessible for future use and auditing processes.

7.6 API-Level Result Summary :

Endpoint	Expected Behavior	Result
GET /api/health	Service availability check	Passed
POST /api/detect-impersonation	Start async scan and return scan_id	Passed
GET /api/scan-status/<scan_id>	Return running/completed states	Passed
GET /api/results/<scan_id>	Return final payload when completed	Passed
GET /api/history	Return previous scans	Passed
POST /api/report	Accept report payload	Passed

Table 1 API-Level Result Summary

7.7 Test Case Results:

Test ID	Scenario	Expected Outcome	Actual Outcome	Status
TC-01	Submit valid Instagram URL	Scan created with scan_id	scan_id returned and processing started	Pass
TC-02	Poll scan status while processing	Status changes pending -> running -> completed	Correct transitions observed	Pass
TC-03	View results after completion	Summary + suspicious candidates displayed	Results rendered correctly	Pass
TC-04	Verify explainability	Reasons shown for flagged profile	Reasons displayed clearly	Pass
TC-05	Check history persistence	Completed scans visible in history	History entries present	Pass
TC-06	Verify DB storage	Profile records stored in MongoDB	Documents visible in instagram_profiles	Pass

Table 2 Test Case Result Summary

7.8 Analytical Discussion

The results show that the FakeShield solution successfully implements its asynchronous explainable system which detects impersonation through its persistent detection capabilities. The system maintains its functionality during live user interactions while generating valuable results which users can use to make decisions. The existing system can be developed into a stronger anti-impersonation intelligence system through its current testing phase and platform extension process.

7.9 Limitations Observed

Despite successful execution, the following limitations were observed:

1. Total scan duration can increase due to external scraper wait time.
2. Current production-ready logic is strongest for Instagram only.
3. Photo similarity behavior depends on DeepFace/runtime dependency availability.
4. Evaluation is currently functional and scenario-based, not yet benchmarked on a labeled large-scale dataset.
5. Accuracy may vary where impersonators intentionally hide profile metadata.

7.10 Future Directions

The next improvement phase should focus on measurable accuracy, scale, and governance:

1. Add labeled-dataset evaluation with precision, recall, and F1-score.
2. Provide richer live progress indicators in UI during long scans.
3. Expand robust support for additional social platforms.
4. Introduce threshold calibration and analyst feedback loop for continuous tuning.
5. Add advanced filtering, pagination, and export options in history.
6. Strengthen privacy/compliance controls such as retention policy automation and deletion workflows.
7. Increase automated test coverage for API contracts and error-handling paths.

7.11 Chapter Conclusion

The results clearly show that FakeShield solution operates an asynchronous impersonation detection system which provides both explainable results and persistent protection against identity theft. The solution maintains its operational strength throughout real-time use while delivering major results that users need to address. The current solution can advance to a more effective anti-impersonation intelligence system through testing based on data and platform development.

8. Conclusion and Recommendations

8.1 Conclusion

The application FakeShield offers comprehensive capabilities for identifying threats related to social media identity theft by detecting impersonation attempts. The system functions through its user interface elements and backend processing functions to obtain user data and evaluate multiple risks while documenting all results. FakeShield demonstrates that investigators can detect suspicious accounts by studying profile data and user behavior patterns.

The final workflow adhered to its intended operational sequence, which included user data input, scanning initiation, scan history logging, candidate selection, risk assessment, development of justification for recommendations, and report generation. The system's asynchronous architecture improved user experience by eliminating front-end delays which would have caused users to experience extended waiting times.

8.2 Achievement Against Objectives

The major objectives defined for this project were achieved as follows:

1. End-to-end scan flow was implemented and validated from input to result.
2. Asynchronous job execution with status polling was successfully integrated.
3. Multi-factor detection logic was applied using bio, username, display-name, behavioral, and photo-based signals.
4. Explainable output was provided through human-readable flagged reasons.
5. MongoDB persistence supported profile, post, result payload, and history retrieval.
6. Frontend modules provided a complete user journey across Home, Scan, Results, and History.

8.3 Key Technical Contributions

The project makes the following practical contributions:

1. A modular detection pipeline that supports extensibility of scoring signals.
2. A non-blocking API-first architecture suitable for long-running analysis tasks.
3. A transparent risk reporting model that assists manual verification instead of replacing it.
4. A reproducible storage-backed workflow that enables retrospective analysis through scan history.
5. A foundation for platform expansion and model calibration in future versions.

8.4 Practical Significance

FakeShield functions as an early detection system which detects manual impersonation in situations where monitoring becomes difficult and requires extensive resources. The system reduces the effort required to find dangerous candidates through its capability of displaying suspicious accounts together with their associated risk labels and explanation. The new feature enhances productivity for content creators and small businesses and digital identity users who require continuous monitoring support.

8.5 Compliance, Ethics, and Responsible Use

The project meets privacy and security requirements for personal data handling according to its governance framework. The project in India focuses on two primary issues which involve identity theft and impersonation according to the IT Act and data processing standards established by the DPDP. The production system implementation requires more robust legal controls which should include a defined retention policy and deletion processes and advanced audit logging and user notification systems that operate at the policy level. The system should operate as a decision-support system which provides assistance to users instead of making automatic final decisions.

8.6 Remaining Gaps

Although functionally successful, the project still has some constraints:

1. Scan completion time can vary due to external scraper latency.
2. Full multi-platform parity is not yet achieved beyond Instagram-centric behavior.
3. Photo-based analysis depends on environment-specific deep-learning dependencies.
4. Large-scale labeled benchmarking is pending for statistically rigorous accuracy claims.
5. Adversarial profile obfuscation can reduce signal reliability in edge cases.

8.7 Recommendations and Future Roadmap

The following steps are recommended for the next phase:

1. Build and evaluate on a labeled dataset with precision, recall, F1-score, and confusion matrix reporting.
2. Add confidence calibration and threshold tuning based on validated outcomes.
3. Introduce richer progress telemetry and partial-result indicators in the user interface.
4. Expand production-grade adapters for additional social platforms.
5. Implement role-based access controls, retention automation, and compliance documentation.
6. Increase automated testing coverage for API contracts, async jobs, and failure recovery paths.
7. Add analyst feedback loops to improve model behavior over time.

9. List of References

- [1] Flask, "Flask Documentation," Pallets Projects. [Online]. Accessible at: <https://flask.palletsprojects.com/>. Last accessed: Apr. 23, 2026.
- [2] React, "React Documentation," Meta. [Online]. Accessible at: <https://react.dev/>. Last accessed: Apr. 23, 2026.
- [3] MongoDB, "MongoDB Documentation," MongoDB, Inc. [Online]. Accessible at: <https://www.mongodb.com/docs/>. Last accessed: Apr. 23, 2026.
- [4] Apify, "Apify Documentation," Apify Technologies s.r.o. [Online]. Accessible at: <https://docs.apify.com/>. Last accessed: Apr. 23, 2026.
- [5] Sentence-Transformers, "Sentence-Transformers Documentation," UKPLab. [Online]. Accessible at: <https://www.sbert.net/>. Last accessed: Apr. 23, 2026.
- [6] SeatGeek, "fuzzywuzzy: Fuzzy String Matching in Python," GitHub repository. [Online]. Accessible at: <https://github.com/seatgeek/fuzzywuzzy>. Last accessed: Apr. 23, 2026.
- [7] Serengil and O. Ozpinar, "DeepFace: A Face Recognition Framework for Python," GitHub repository. [Online]. Accessible at: <https://github.com/serengil/deepface>. Last accessed: Apr. 23, 2026.
- [8] Python Software Foundation, "Python Documentation," Python.org. [Online]. Accessible at: <https://docs.python.org/3/>. Last accessed: Apr. 23, 2026.
- [9] Flask-CORS, "Flask-CORS Documentation," GitHub repository. [Online]. Accessible at: <https://flask-cors.readthedocs.io/>. Last accessed: Apr. 23, 2026.
- [10] Meta, "Instagram Help Center," Meta Platforms, Inc. [Online]. Accessible at: <https://help.instagram.com/>. Last accessed: Apr. 23, 2026.
- [11] Government of India, "The Information Technology Act, 2000." [Online]. Accessible at: <https://www.indiacode.nic.in/>. Last accessed: Apr. 23, 2026.
- [12] Government of India, "Digital Personal Data Protection Act, 2023." [Online]. Accessible at: <https://www.indiacode.nic.in/>. Last accessed: Apr. 23, 2026.

10. Plagiarism Report

Major Project		
ORIGINALITY REPORT		
9%	6%	2%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS
		7%
		STUDENT PAPERS
PRIMARY SOURCES		
1	Submitted to National Forensic Sciences University Student Paper	5%
2	Submitted to Glyndwr University Student Paper	<1%
3	docshare.tips Internet Source	<1%
4	etd.aau.edu.et Internet Source	<1%
5	"Innovations and Interdisciplinary Solutions for Underserved Areas", Springer Science and Business Media LLC, 2025 Publication	<1%
6	Submitted to University of Staffordshire Student Paper	<1%
7	Submitted to Islington College,Nepal Student Paper	<1%
8	Submitted to UNITEC Institute of Technology Student Paper	<1%
9	Submitted to Georgia Institute of Technology Main Campus Student Paper	<1%
10	www.frontiersin.org Internet Source	<1%
11	Submitted to Technological Institute of the Philippines Student Paper	<1%

12	ijsrem.com Internet Source	<1 %
13	su-plus.strathmore.edu Internet Source	<1 %
14	Daniel Wilianto, Abba Suganda Girsang. "Automatic Short Answer Grading on High School's E-Learning Using Semantic Similarity Methods", TEM Journal, 2023 Publication	<1 %
15	indah.ump.edu.my Internet Source	<1 %
16	ucf.digital.flvc.org Internet Source	<1 %
17	Thanet Markchom, Huizhi Liang, James Ferryman. "Review of Explainable Graph-Based Recommender Systems", ACM Computing Surveys, 2025 Publication	<1 %
18	Submitted to University of Teesside Student Paper	<1 %
19	dpoclub.in Internet Source	<1 %
20	repository.sdu.edu.kz Internet Source	<1 %
21	lawslearned.com Internet Source	<1 %
22	www.mdpi.com Internet Source	<1 %
23	Meryem Chajia, El Habib Nfaoui. "Customer Churn Prediction Approach Based on LLM Embeddings and Logistic Regression", Future Internet, 2024 Publication	<1 %



24	www.aui.ma Internet Source	<1 %
25	nebula.indocisc.co.id Internet Source	<1 %
26	scalar.usc.edu Internet Source	<1 %
27	Mark S. Merkow. "Secure, Resilient, and Agile Software Development", CRC Press, 2019 Publication	<1 %

Exclude quotes Off
Exclude bibliography Off

Exclude matches Off

